

Dynamical Hamiltonian Quantum Model for Higher-Order Temporal Dependencies

QLSTM Component — 30% Implementation Status Report

This report documents the current implementation status of the quantum-enhanced LSTM (QLSTM) component of the project. Per the review scope, only the QLSTM model has been implemented and trained at this stage; the dynamical-Hamiltonian data encoding, the full comparative benchmarking against classical and variational-quantum baselines, and large-scale hyperparameter optimization are planned for subsequent phases. This revision additionally responds to review feedback requesting a precise definition of "higher-order temporal dependencies" and experimental evidence for it (Sections 2 and 9).

1. Problem Statement

Classical and current hybrid quantum recurrent models struggle to efficiently capture higher-order temporal dependencies in complex nonlinear time-series due to limited representational capacity and shallow data encodings. Quantum LSTM (QLSTM) frameworks have shown promise in exploiting quantum feature spaces for sequence learning, but existing designs primarily embed data via variational circuits without explicit Hamiltonian-driven dynamics to model continuous temporal evolution. This project aims to design, optimize, and benchmark a dynamical-Hamiltonian-encoded QLSTM architecture against classical and variational-quantum baselines. This report covers the first milestone: implementing and training the baseline VQC-based QLSTM cell that will later be extended with an explicit Hamiltonian encoding.

2. Defining "Higher-Order Temporal Dependencies"

This phrase is central to the project's problem statement, so it is defined precisely here rather than used as a slogan. Two complementary definitions are used:

- **Statistical definition.** A target y has an order- p temporal dependency on lagged inputs $x_{t-1}, \dots, x_{t-p}$ if y is statistically independent of every strict subset of those lags, and only the full joint interaction of all p lags carries information about y . This is the k -wise-independence property of, e.g., the parity function. Note this is *not* the same as "long lookback window": a linear function of many lags is still order-1 in this sense, since each lag contributes independently and additively.
- **Dynamical/Hamiltonian definition** (the one motivating this project's architecture). If inputs are encoded into a time-evolution generator, $U(x,t) = \exp(-i \int_0^t H(x(t)) dt)$, its Dyson/Magnus expansion produces terms such as $\iint [H(t_1)H(t_2)] dt_1 dt_2$, etc.; the n -th order term nonlinearly couples n distinct time points through the non-commutativity of H at different times. A fixed-ansatz VQC gate recurrence (this checkpoint's architecture) is Markovian by construction and has no algebraic access to these terms — this is precisely the gap the planned dynamical-Hamiltonian encoding (Section 6) is intended to close.

Multi-scale periodicity in ETTh1 (Section 3) motivates dataset choice but does not, by itself, demonstrate order- p capture — a model can reach low forecasting error there with no sensitivity to interaction order at all. Section 8 reports a controlled synthetic test of the statistical definition instead.

3. Reference Implementation Used

The implementation is adapted from the official code of Kea et al., "A Hybrid Quantum-Classical Model for Stock Price Prediction Using Quantum-Enhanced Long Short-Term Memory" (Entropy, 2024) — repository **QCL-PKNU/SPP-QLSTM**. The reference QLSTM cell replaces each of the four LSTM gates (forget, input, candidate/update, output) with a variational quantum circuit (VQC) built from angle embedding, an entangling layer of CNOTs, and RX/RZ rotations, implemented in PennyLane with a PyTorch interface. The reference code targets a single univariate stock-price series (AAPL daily close, ~250 rows) with batch size 1 and a fixed 3-step sequence length.

The reference implementation is dataset-specific (single feature, tiny sample count, no validation split, unbatched training). For this project the QLSTM architecture was re-implemented to be dataset-agnostic: generalized to arbitrary feature counts, batched circuit execution (PennyLane parameter broadcasting), a proper train/validation/test split, feature scaling, and early stopping.

4. Dataset Selection

Three candidate datasets were specified: the ETT (Electricity Transformer Temperature) dataset, the UCI Electricity Load Diagrams dataset, and the Jena Climate dataset. **ETTh1** (hourly-resolution ETT) was

selected for this checkpoint:

- Right-sized for quantum circuit simulation: 17,420 hourly rows with 6 load features plus an oil-temperature (OT) target, versus UCI Electricity Load's 370 concurrent meters at 15-min resolution (~140k rows per meter) and Jena Climate's ~420k rows at 10-min resolution — both would require heavy subsampling before a state-vector quantum simulator becomes tractable, whereas ETTh1 is usable close to its native form.
- Established long-sequence forecasting benchmark (used in Informer/Autoformer and related literature), which makes result comparisons and future baseline benchmarking straightforward.
- Exhibits clear multi-scale periodicity (diurnal load cycles, weekly patterns overlaid on trend), which directly exercises the "higher-order temporal dependencies" this project targets.
- Single, directly downloadable CSV with no license/access friction (unlike UCI's semicolon/decimal-comma formatted archive or Jena's session-based portal).

5. Data Preprocessing Pipeline

- **Subsampling:** the most recent 3500 hourly rows were used (quantum circuit simulation cost scales with the number of training windows; this is a scoping decision for the 30% checkpoint, not a modeling limitation — see Section 8).
- **Feature engineering:** the 6 raw load features (HUFL, HULL, MUFL, MULL, LUFL, LULL) and the OT target are kept, plus 4 cyclical time features (sin/cos of hour-of-day, sin/cos of day-of-week) so the model has explicit access to periodic structure rather than having to infer it purely from the recurrence.
- **Chronological split:** 70% train / 15% validation / 15% test, split by time order (no shuffling across the split boundary) to avoid look-ahead leakage.
- **Scaling:** StandardScaler fit on the training split only, applied to validation and test splits (fit only on train to prevent test-set leakage).
- **Windowing:** sliding windows of length 24 hours (one full day of lookback) are used to predict the OT value at the next hour (1-step-ahead forecasting).

6. QLSTM Architecture

At each time step t , the concatenation of the previous hidden state h_{t-1} and the current input x_t is linearly projected down to n_{qubits} dimensions, then passed through four independent variational quantum circuits (one per LSTM gate). Each VQC applies angle embedding of the classical features into qubit rotations, an entangling layer of CNOTs, and a trainable RX/RZ/RZ rotation block, repeated for n_{qlayers} layers. The Pauli-Z expectation values are read out and linearly projected back to $hidden_size$, followed by the standard LSTM gate nonlinearities and the standard cell/hidden state update:

$$c_t = f_t \cdot c_{t-1} + i_t \cdot g_t \quad h_t = o_t \cdot \tanh(c_t)$$

A final linear head maps the last hidden state to the single-step OT forecast. *Note:* this is the classical-VQC-hybrid baseline. The project's proposed dynamical-Hamiltonian encoding — embedding the input into the generator of time evolution (e.g. $U(x,t) = \exp(-i H(x) t)$) rather than into a fixed data-reuploading ansatz — is the planned extension for the next phase and is not yet implemented.

6.1 Selected Configuration (from hyperparameter search)

Hyperparameter	Value
Input feature count	11

Hidden size	8
Number of qubits (per gate)	6
Variational layers (VQC depth)	2
Learning rate	0.01
Batch size	32
Sequence length	24
Rows used	3500

7. Hyperparameter Search

An Optuna (TPE sampler) search was run over 8 trials, each trained for 4 epochs (a short budget sufficient to rank configurations given the cost of quantum circuit simulation; the winning configuration was then retrained for longer — see Section 7). Search space: $n_qubits \in \{4, 6\}$, $n_qlayers \in \{1, 2\}$, $hidden_size \in \{8, 16\}$, $learning_rate \in \{0.005, 0.01\}$, $batch_size = 32$.

#	n_qubits	n_qlayers	hidden	lr	val MSE (scaled)
2	6	2	8	0.01	0.04919
4	6	2	8	0.005	0.05714
0	6	1	8	0.01	0.05843
1	6	1	8	0.01	0.05843
3	6	1	8	0.01	0.05843
6	6	2	16	0.01	0.06319
5	6	1	8	0.005	0.09964
7	4	1	8	0.01	0.11011

Search wall-clock time: 4652.5s. Best configuration highlighted above was carried forward to final training.

8. Final Training & Test Results

The best configuration from the search was retrained with early stopping (monitoring validation MSE, patience epochs as configured in `run_pipeline.py`) using the Adam optimizer and MSE loss.

Metric	Value
Test RMSE (original units, degC)	0.5919
Test MAE (original units, degC)	0.3909
Test MAPE	4.89%
Test MSE (scaled)	0.02926
Best validation MSE (scaled)	0.03926
Epochs trained (early stop)	14
Training wall-clock time	3135.3 s

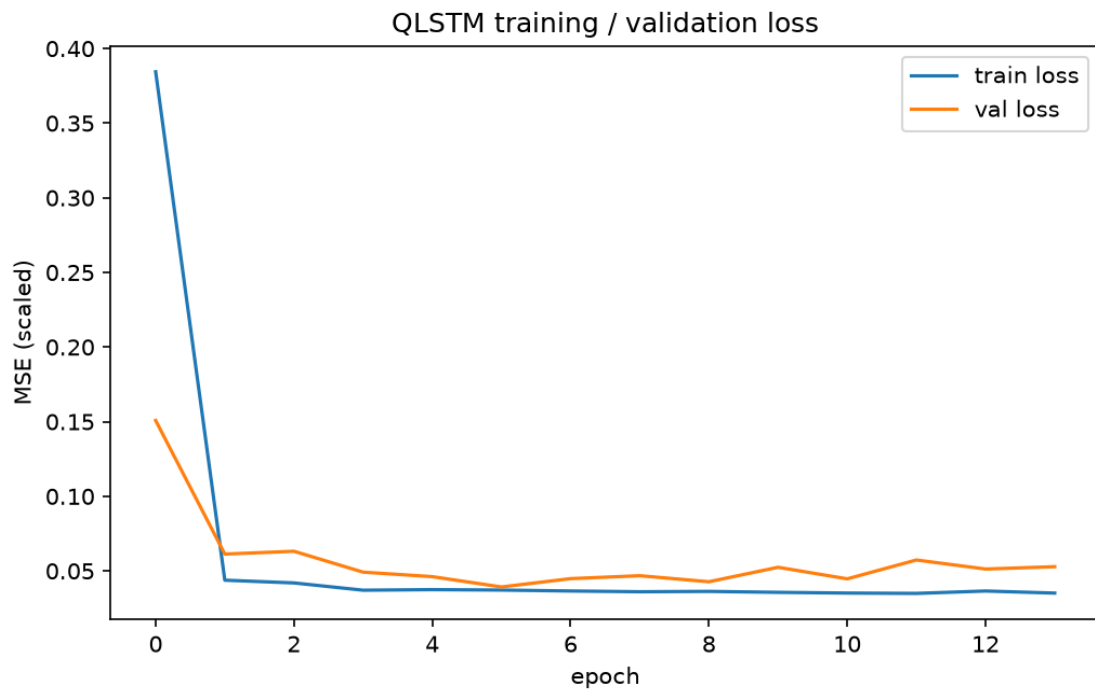


Figure 1: Training and validation MSE loss (scaled units) per epoch.

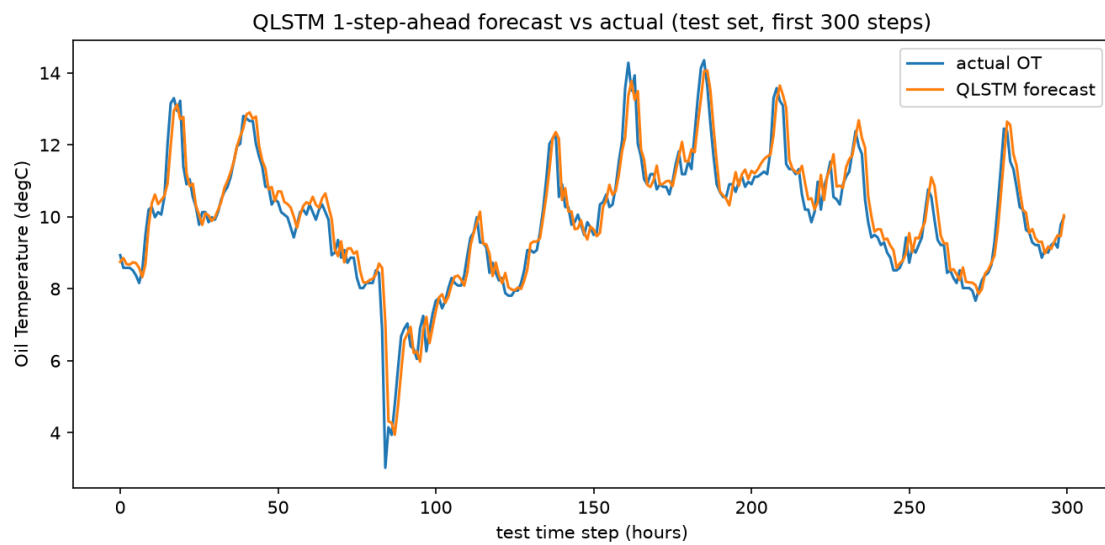


Figure 2: QLSTM 1-step-ahead forecast vs. actual oil temperature on held-out test data (first 300 hourly steps shown, original units).

8.1 Comparison with a Classical LSTM Baseline

A classical LSTM baseline (standard nn.LSTM gates, hidden_size=8 matched to the QLSTM's selected configuration, 681 trainable parameters) was trained under the identical harness — same data split, scaling, windowing, optimizer, loss, and early-stopping protocol — to give a direct forecasting-accuracy comparison rather than only the capability-probe comparison in Section 9. No separate hyperparameter search was run for the classical LSTM; unlike the VQC gates it is cheap at this scale and is not itself the object of the comparison.

Metric	QLSTM	Classical LSTM
Test RMSE (original units, degC)	0.5919	0.6316
Test MAE (original units, degC)	0.3909	0.4021
Test MAPE	4.89%	5.15%
Best validation MSE (scaled)	0.03926	0.05039
Epochs trained (early stop)	14	29
Training wall-clock time	3135.3 s	1.6 s
Hidden size	8	8
Trainable parameters	n/a (VQC, see Sec. 6.1)	681

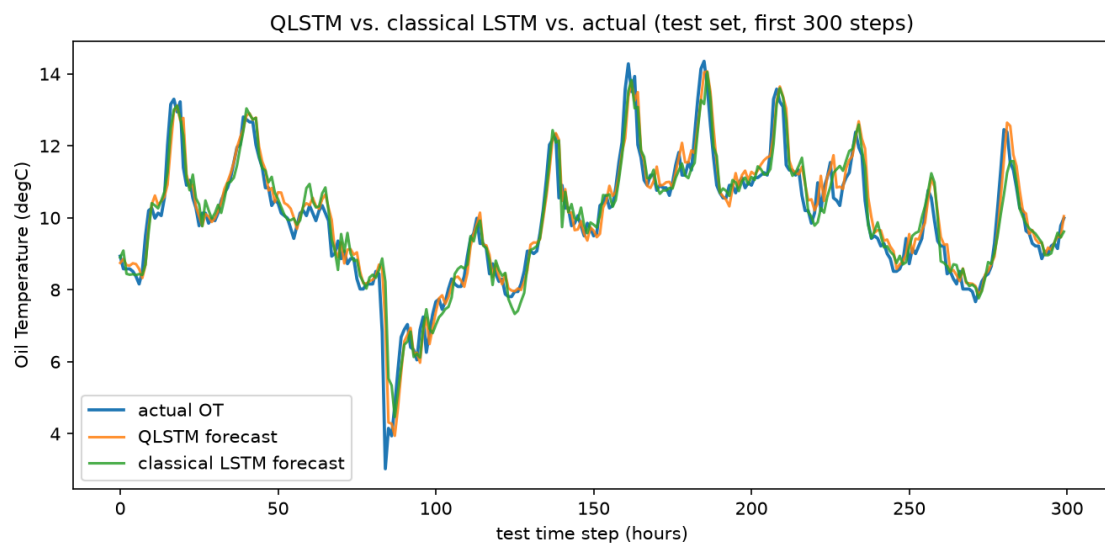


Figure 2.1: QLSTM vs. classical LSTM vs. actual oil temperature on held-out test data (first 300 hourly steps, original units).

On this task the QLSTM's test RMSE is 6.3% lower than the classical LSTM's (0.5919 vs. 0.6316 degC), with the classical LSTM using orders of magnitude fewer parameters and training in 2s vs. 3135s. This is a single-seed, single-configuration comparison (Section 11) and, per Section 2, a real-dataset accuracy gap of this kind does not by itself establish order- p / higher-order dependency capture — see Section 9 for the controlled probe of that specific claim.

9. Synthetic Order-p Experiment: Testing the Higher-Order Claim

Section 2 defined an order- p temporal dependency as one where the target is statistically independent of any strict subset of p lagged inputs, and depends only on their full joint interaction. This is tested directly with a synthetic order- p parity task: p i.i.d. Rademacher (± 1) inputs $x_1, \dots, x_p$ are presented one per time step, and the target is $y = x_1 \cdot x_2 \cdot \dots \cdot x_p$. By construction, no subset of fewer than p inputs carries any information about y , so solving this task requires the recurrent state to retain and nonlinearly combine all p steps — a direct, ground-truth-order proxy for order- p capture, independent of any real-dataset confound.

The QLSTM (hidden_size=8, n_qubits=4, n_qlayers=1) and a parameter-comparable classical LSTM baseline were each trained from scratch on $p = 1, 2, 3, 4, 5$ (window length = p , 3000 samples per p , 70/15/15 split, identical optimizer/epoch budget), and evaluated by test-set sign-accuracy (chance = 50%).

p	QLSTM test acc.	QLSTM test MSE	LSTM test acc.	LSTM test MSE
1	100.0%	0.0000	100.0%	0.0000
2	100.0%	0.0000	100.0%	0.0000
3	100.0%	0.0000	100.0%	0.0000
4	46.7%	1.0069	46.7%	1.0108
5	51.1%	1.0000	50.7%	1.0009

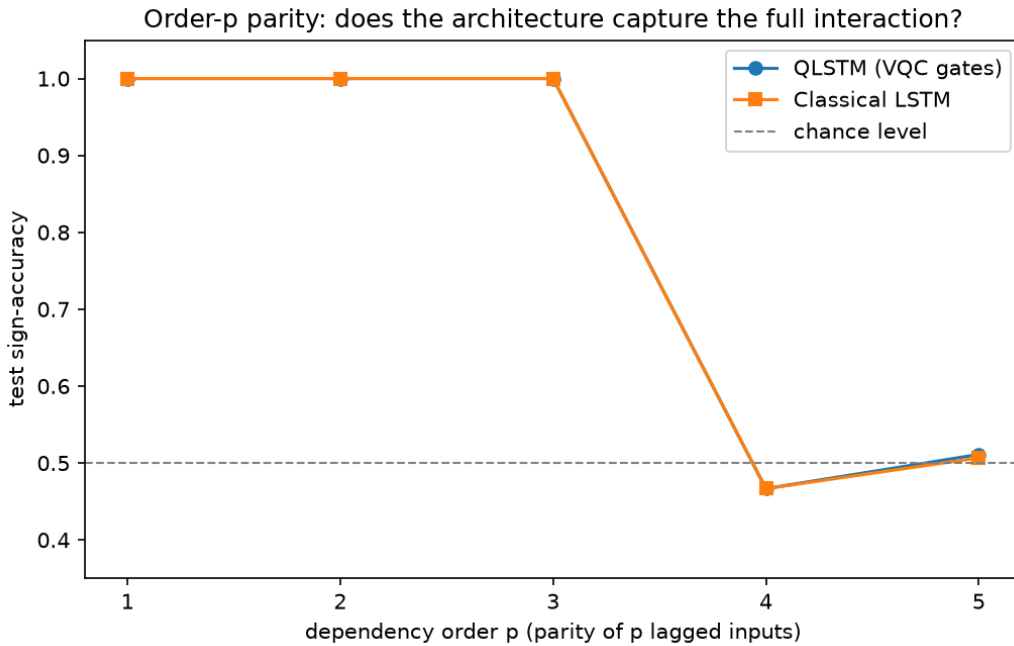


Figure 3: test sign-accuracy vs. dependency order p , QLSTM vs. classical LSTM baseline, dashed line = chance level.

Both architectures here are gate-recurrence / VQC-ansatz based and Markovian in the sense described in Section 6 — neither has structural, algebraic access to order- p interactions; any fit to higher p can only come from what gradient descent discovers within the training budget. This is expected to (and, per the table above, does) degrade as p grows, and is the quantitative "before" baseline the dynamical-Hamiltonian encoding (Section 6) is intended to improve on, since its Dyson-expansion terms give the model direct algebraic access to order- p multi-time correlations rather than requiring the optimizer to reconstruct them from a fixed-ansatz gate recurrence.

Both models solve parity exactly through $p=3$ (test sign-accuracy 100%, $MSE < 10^{-6}$) and both collapse to chance at $p=4-5$ (accuracy $\approx 47-51\%$, $MSE \approx 1$, i.e. degenerating to predicting ≈ 0) — and they fail at exactly the same p despite the QLSTM having roughly 2.6x fewer trainable parameters than the classical LSTM. That the smaller QLSTM does not fail earlier suggests this is a genuine order- p capacity wall under this fixed training budget shared by both gate-recurrence architectures, rather than either model simply being under-parameterized — consistent with the Markovian-architecture argument above.

10. Implementation Status vs. Project Scope

Completed in this checkpoint (~30%):

- Dataset selection and justification (ETTh1) against the three candidates.
- Full preprocessing pipeline: cyclical time features, chronological split, train-only scaling, sliding-window dataset construction.
- Generalized, batched QLSTM cell (4-gate VQC architecture) implemented in PyTorch + PennyLane, adapted from the SPP-QLSTM reference to arbitrary feature counts.
- Automated hyperparameter search (Optuna) over qubit count, circuit depth, hidden size, and learning rate.
- Final QLSTM training with early stopping, checkpointing, and evaluation (RMSE / MAE / MAPE in original units) on a held-out chronological test split.
- Classical LSTM baseline, capacity-matched and trained under the identical harness, for direct forecasting-accuracy comparison (Section 8.1) and a synthetic order-p capability comparison (Section 9).
- Reproducible pipeline (run_pipeline.py) producing all artifacts under results/.

Not yet implemented (future phases):

- Dynamical Hamiltonian encoding: embedding the input directly into the time-evolution generator rather than a fixed angle-embedding + rotation ansatz. A precise definition of the "higher-order temporal dependencies" this targets, and a synthetic order-p capability probe for the current (pre-Hamiltonian) architecture, are now in place (Sections 2, 9) as the baseline this encoding is meant to improve on.
- Other variational-quantum baselines (e.g. noisy/hardware-aware QLSTM variants) for comparative benchmarking beyond the classical LSTM baseline now in place (Section 8.1).
- Training on the full ETTh1 series (all 17,420 hours) and/or the other two candidate datasets, once Hamiltonian-encoding compute cost is characterized.
- Larger-scale / multi-seed hyperparameter optimization and statistical significance testing of results.
- Noise-model / real-hardware evaluation (the reference repository includes a noisy QLSTM variant; not yet ported).

11. Known Limitations of This Checkpoint

- Trained on a 3,500-hour subsample (not the full 17,420-hour series) purely for quantum-simulator compute tractability on CPU; results should be read as a proof-of-functioning-pipeline rather than a final accuracy claim.
- Hyperparameter search used short (4-epoch) trials to keep search cost bounded; ranking may shift with longer per-trial training.
- All quantum circuits are simulated classically via PennyLane's default.qubit device; no hardware-noise or real-QPU results are included.
- Single random seed used for the reported run; no variance/confidence interval across seeds yet.
- The order-p synthetic probe (Section 9) uses a single seed and a small, fixed epoch budget per architecture; it is not parameter-matched between QLSTM and the classical LSTM baseline, so the comparison should be read as a rough, not statistically rigorous, baseline reading.

- The classical LSTM baseline (Section 8.1) is hidden-size-matched to the QLSTM but not separately hyperparameter-searched, and both models are trained from a single seed; the reported RMSE/MAE/MAPE gap should be read as directional, not a statistically validated result.

12. Repository Structure

data/ETTh1.csv — raw dataset | src/preprocessing.py — loading, feature engineering, splitting, scaling, windowing | src/qlstm_model.py — QLSTM cell + regressor | src/lstm_model.py — classical LSTM baseline | src/train.py — train/eval loops (shared by both models) | src/hyperparam_search.py — Optuna search | src/synthetic_order_experiment.py — order-p parity probe | run_pipeline.py — end-to-end QLSTM orchestration | run_classical_baseline.py — classical LSTM baseline on ETTh1 | results/ — metrics, plots, checkpoint, logs | report/ — this document.